

Jason Song
CMSC 471
FB79801

PA1

1.1-1.4 (See dc.py)

1.5

```
$ python dcsolve.py dog ca
goal word not a 3 or 4 letter word
Traceback (most recent call last):
```

```
$ python dcsolve.py why ask steps
dc(why,ask,steps) cost:5.00; time:0.001; solution:why wha aha ahs ass ask; deltas:[-2, -1, -1, 0.0, 0.0]
```

```
$ python dcsolve.py dog cat
dc(dog,cat,steps) cost:3.00; time:0.000; solution:dog cog cot cat; deltas:[0.0, 0.0, 0.0]
```

```
$ python dcsolve.py why ask
dc(why,ask,steps) cost:5.00; time:0.001; solution:why wha aha ahs ass ask; deltas:[-2, -1, -1, 0.0, 0.0]
```

```
$ python dcsolve.py why ask
dc(why,ask,steps) cost:5.00; time:0.001; solution:why wha aha ahs ass ask; deltas:[-2, -1, -1, 0.0, 0.0]
```

```
$ python dcsolve.py test bush
dc(test,bush,steps) cost:3.00; time:0.000; solution:test best bust bush; deltas:[0.0, 0.0, 0.0]
```

```
$ python dcsolve.py love hate
dc(love,hate,steps) cost:3.00; time:0.000; solution:love hove have hate; deltas:[0.0, 0.0, 0.0]
```

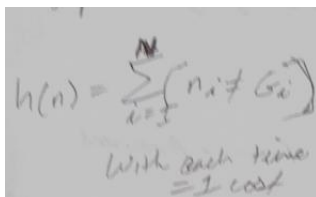
```
$ python dcsolve.py bear duck steps
dc(bear,duck,steps) cost:4.00; time:0.000; solution:bear beak beck buck duck; deltas:[0.0, 0.0, 0.0, 0.0]
```

```
$ python dcsolve.py bear duck scrabble
dc(bear,duck,scrabble) cost:11.00; time:0.000; solution:bear beak beck buck duck; deltas:[0.0, 0.0, 0.0, 0.0]
```

```
$ python dcsolve.py bear duck frequency
dc(bear,duck,frequency) cost:38.24; time:0.704; solution:bear beak beck buck duck; deltas:[-34.24, -22.776, -14.264, -7.23]
```

2.1

Steps:


$$h(n) = \sum_{i=0}^n (n_i \neq G_i)$$

With each time = 1 cost

Summation of letters not equal to the goal word letters.

The unavoidable cost is equal to the number of letters within the start state word that isn't the same as the goal state word letter within that position. So in the example of WHY -> ASK, all 3 letters in Y need to be changed at least once to reach their goal state and unavoidable cost would be 3.

Reason why you can't overestimate the true optimal remaining cost is because if there are x amount of mismatched letters and each action costing 1, each letter change can only have the same amount of actions. Thus heuristics is admissible.

Scrabble:

$$h(n) = \sum_{\text{letter} \neq \text{goal}} S(x_{\text{cost}})$$

$x = \text{letter cost}$

Summation of start letter * scrabble cost of letter

The unavoidable cost for scrabble type cost map is the cost of the letter of the goal letters. If for example, it were to change the letter immediately to the goal state word letters after 1 turn, it would equal the letter costs of the goal state letters. This can vary depending on the goal state word that we are trying to reach.

It can't overestimate true optimal remaining cost $h(n)$ because with each action changing 1 letter at some point changing it to the goal letter. Any steps in between need to be scored which means it can be greater but can't overestimate the lowest, optimal, remaining scrabble costs to reach the goal.

Frequency:

$$h(n) = 1 + \text{rarity}(w)$$

The unavoidable cost for frequency is given by the action cost given in the directions as $1 + \text{rarity}(w)$. Because the word can't be exactly the same, there is hypothetically at least 1 letter that is different from the goal word. Since the action cost of a frequency cost method $1 + \text{rarity}(w)$ means that there will at least be this amount of cost.

It can't be overestimated because same principle as the others, because each action is a result of changing per letter position, each letter change needs to have same or greater amount of cost than the optimal which would be $1 + \text{rarity}(w)$ and that is capped at the lower bound.

Part 2.x.x

PAIR	COST	UCS COST	UCS EXP	A* COST	A* EXP	REDUCTION
WHY -> ASK	steps	5	773	5	16	97.93%
WHY -> ASK	scrabble	9	765	9	13	98.30%
WHY -> ASK	frequency	38.855	808	38.855	755	6.56%
BEAR -> DUCK	steps	4	691	4	6	99.13%
BEAR -> DUCK	scrabble	11	2981	11	6	99.80%
BEAR -> DUCK	frequency	38.24	1084	38.24	720	33.58%

2.2.1

Yes it returned the same optimal cost.

2.2.2 Largest Reduction:

WHY -> ASK: scrabble (98.30% reduction)

BEAR -> DUCK: scrabble (99.80% reduction)

2.2.3 Least informative:

Found adjacent pair:

the -> she

rarity(the) = 0.016226212

rarity(she) = 2.0556157

Forward cost = 3.0556157

Reverse cost = 1.016226212

This demonstrates direction-dependent frequency costs.

2.2.4

UCS differs because it uses the cost already traversed and accumulated aka $g(n)$. While A^* uses $g(n)+h(n)$ that applies the estimated remaining cost. In step cost for example, not only does it count where the costs are currently, but the number of letters that still haven't reached the goal state in per letter for the goal word. Where as using heuristics for scrabble, estimates consist of the goal letter costs and frequency using the $1+\text{rarity}(w)$ as the heuristic. With this, A^* can use the lowest cost at that particular time as well as estimate based on heuristic what would be lower cost moving forward. This estimation results in lower expanded nodes, while UCS traverses every potential legal path.

2.3

Steps:

Yes because every action costs 1 and reversal preserves the number of actions.

Changing one letter costs exactly 1 in either direction, so every reversed path has the same number of actions and therefore the same cost.

Scrabble:

No because the action can be reversed, however the cost is based on the letter that's being inserted based on the scrabble cost map.

For example, the -> tae. To move forward a letter we go from h to a. would cost 1 however, from a to h would cost us 4.

Frequency:

No because similar to the scrabble reasoning, this time the rarity score would be a factor.

For example, the word 'the' has a rarity of .01622611212. where the word 'she' 2.0556157. The cost to go from 'the' the 'she' would cost the rarity score of the new word, which would be $1+\text{rarity}$ of the new word. So going from the -> she, would be $1+2.0556157 = 3.0556157$. In reverse, she->the, would be $1+.016226212 = 1.016226212$

Question 4.

4.1

Gemini 3.6 Thinking

In DOGCAT, the initial and goal states are lowercase words of the same length—three or four letters—and every legal state must appear in the supplied dictionary. A legal action changes exactly one letter. Under the frequency objective, entering word w costs $1 + \text{rarity}(w)$, where the nonnegative rarity value is stored in the dictionary.

The baseline heuristic h_0 (n) is the number of positions in which state n differs from the goal. Propose a rarity-aware heuristic intended to be more informative than h_0 while remaining admissible. Define it precisely and justify your proposal.

By "more informative," we mean that the proposed heuristic should be at least as large as h_0 for every state and strictly larger for at least one state, without overestimating the true optimal remaining cost.

4.2

Definition of $h_r(n)$

Let $r_{\min} = \min_{w \in D} \text{rarity}(w)$ be the minimum rarity value present across the dictionary D .

For any state n , let $h_0(n)$ be the Hamming distance between n and the goal g (i.e., the number of character positions where n and g differ). The proposed heuristic $h_r(n)$ is defined as:

$$h_r(n) = h_0(n) \cdot (1 + r_{\min})$$

4.3

Yes the heuristic equation is admissible because if $h_0(n)$ = the number of letters in the start word, it will have, at minimum, the same amount of actions and thus the associated costs with $1 + \text{rarity}(w)$ of those changes to potential reach the goal word.

$h_r(n) = h_0(n) \cdot (1 + r_{\min})$

If n is H away from goal

Cost $\rightarrow (1 + r_{\min})$

$h_0(n) \rightarrow$ Counter away from goal

$h_0(n) \rightarrow$ How many letters in word is not the goal letter

	Action
W $\rightarrow$ A	+1
H $\rightarrow$ S	+1
Y $\rightarrow$ K	+1

Each of the letters in W-H-Y require at least one change each

$h_0(n) = 3$

4.4

The proposed heuristic is valid.

4.5

PAIR	COST	Old h(n)	Old exp	New h(n)	New Exp	REDUCTION
WHY -> ASK	frequency	38.855	755	38.855	634	16.03
BEAR -> DUCK	frequency	38.24	720	38.24	302	58.06

The solution cost is the same validating that it works as intended. And it looks like the new heuristics has made a difference in reducing the number of expanded nodes and using the proposed heuristics would help with efficiency.

Appendix AI use:
Chatgpt

- to help with instance outputs.
 - verified using test inputs to trigger error prompts.
- summary of the parameters used for each function
 - get a understanding what each parameter is and their purpose.
- understand iteration of action function
 - verified through use of print outputs of state position letter and new state throughout iteration

Gemini:

- initial response from question 4 prompt for recommended heuristic that contained heuristic breakdown and it's explanation to help determine its admissibility.
- used it to break down basic logic of admissibility with example for answering part 2: question 3 and 5.